

1 SECTION 1 — ASYNC MENTAL MODEL (NON-NEGOTIABLE)

Scope: This section rebuilds how you *think* about execution. No syntax mastery matters if this mental model is wrong.

1. Sync vs Async — Real-Life Analogy (Rewired)

1 Mental Model (Feynman)

Sync = single worker

- One worker.
- One task at a time.
- Worker waits during idle time.

Async = task coordinator

- One worker brain.
- Many tasks in progress.
- Worker switches whenever a task is waiting.

Async does **not** create more workers. Async eliminates *waiting waste*.

2 Why This Exists (Real World Pain)

Scraping pain with sync:

- Network waits dominate runtime.
- CPU sits idle while server thinks.
- 90% time = waiting, not working.

Async exists to:

- Overlap waiting time.
 - Keep CPU busy while network sleeps.
 - Increase throughput without more hardware.
-

3 Minimal Correct Syntax (Context Only)

```
# Sync
html = requests.get(url).text # CPU blocked here

# Async
html = await session.get(url) # CPU released during wait
```

Key difference:

- **Blocked** vs **Released back to event loop**
-

4 Failure Modes

Wrong belief:

- “Async runs things in parallel like threads”

Reality:

- Async runs **interleaved**, not parallel
- CPU-heavy code blocks everything

If you misinterpret this:

- You overload servers
 - You freeze event loop
 - You get banned or stuck runs
-

5 Thinking Exercise

Question:

If one request takes 2 seconds, how long for 10 URLs?

- Sync → ~20 seconds
- Async (ideal I/O bound) → ~2–3 seconds

Now add:

- Rate limits
- Slow servers
- Failures

Your mental model must survive *non-ideal* cases.

6 Debug This Code (Conceptual)

```
async def scrape(urls):  
    for url in urls:  
        await fetch(url)
```

Question:

- Is this async?
- Why is it still slow?

Answer:

- Await inside loop = sequential waiting
 - Async syntax $\neq$ async behavior
-

2. Blocking I/O — Explained Without Myths

1 Mental Model

Blocking I/O = hostage situation

- CPU asks server: "Give me data"
- CPU forced to wait
- Nothing else runs

Async I/O = callback promise

- CPU: "Tell me when ready"
 - CPU does other work
 - Event loop resumes later
-

2 Why This Exists

Web scraping is:

- Network-bound
- Unpredictable
- Latency-heavy

Blocking I/O wastes:

- CPU cycles

- Time
 - Money (cloud costs)
-

3 Minimal Correct Example

```
# Blocking
time.sleep(2)

# Non-blocking equivalent idea
await asyncio.sleep(2)
```

Difference:

- `time.sleep` freezes everything
 - `asyncio.sleep` yields control
-

4 Failure Modes

Common disaster:

```
async def task():
    time.sleep(5) # blocks entire event loop
```

Outcome:

- All async tasks freeze
 - Appears “stuck”
 - No error shown
-

5 Thinking Exercise

Decide:

- Is DNS lookup blocking?
- Is HTML parsing blocking?
- Is JSON decoding blocking?

Correct answer:

- Network → async

- CPU parsing → sync
-

6 Debug This Code

```
async def fetch(url):  
    response = requests.get(url)  
    return response.text
```

Why broken:

- Sync request inside async function
 - Event loop blocked
 - Fake async
-

3. Event Loop — Traffic Police (Critical Concept)

1 Mental Model

Event loop =

- Central scheduler
- Knows which tasks are waiting
- Decides who runs next

It does **not**:

- Speed up CPU
- Create threads
- Do magic

It **only**:

- Switches tasks at await points
-

2 Why This Exists

Without event loop:

- No coordination
- No fairness
- No scalability

Async needs:

- Deterministic control
 - Predictable switching
 - Central authority
-

3 Minimal Correct Syntax

```
asyncio.run(main())
```

Meaning:

- Create event loop
 - Register tasks
 - Start scheduling
 - Clean shutdown
-

4 Failure Modes

Illegal thinking:

- “Await pauses function like sleep”

Reality:

- Await hands control to event loop
- Event loop decides next runnable task

Misuse leads to:

- Starvation
 - Deadlocks
 - Frozen pipelines
-

5 Thinking Exercise

Question:

What happens if no task is awaiting?

Answer:

- Event loop idle
- CPU sleeps

- Program exits if no pending tasks
-

6 Debug This Code

```
async def main():  
    fetch(ur11)  
    fetch(ur12)
```

Bug:

- Coroutines created
 - Never awaited
 - Never executed
-

4. What Async CAN and CANNOT Speed Up

1 Mental Model

Async speeds up:

- Network calls
- Disk I/O
- Waiting-heavy operations

Async does NOT speed up:

- HTML parsing
 - Regex
 - JSON decoding
 - CPU loops
-

2 Why This Matters

Misuse causes:

- Slower performance
- Complex bugs
- False benchmarks

Correct boundary = survival skill.

3 Minimal Illustration

```
# Good async
await session.get(url)

# Bad async
await parse_html(html) # CPU work
```

4 Failure Modes

Symptoms:

- Async slower than sync
- High CPU usage
- No throughput gain

Root cause:

- CPU-bound logic inside event loop
-

5 Thinking Exercise

Classify:

- Fetch → async
 - Parse → sync
 - Save to DB → depends
 - Logging → sync but lightweight
-

6 Debug This Code

```
async def pipeline(url):
    html = await fetch(url)
    data = await parse(html)
    return data
```

Bug:

- `parse` should not be async
- No await boundary needed

- Event loop blocked

SECTION 1 OUTPUT ARTIFACT

Mental Timeline (Text Diagram)

```

SYNC:
[Req1 wait] → [Parse] → [Req2 wait] → [Parse] → [Req3 wait]

ASync:
[Req1 wait]
[Req2 wait]
[Req3 wait]
→ [Parse1][Parse2][Parse3]

```

Hinglish Explanation (Mandatory)

Sync scraping ek mazdoor hai jo ek kaam khatam hone tak baaki sab kaam chhod deta hai. Async scraping traffic controller hai — jo dekhta hai kaun ruk raha hai, kaun chal sakta hai. Speed ka game nahi hai, control ka game hai.

One-Line Truth (Lock This In)

Async is not about doing more work. Async is about not wasting time while waiting.

Section 1 Pass Criteria (Self-Check)

If you still think:

- async = faster by default ❌
- async = parallel ❌
- async = threads ❌

You failed Section 1.

If you think:

- async = wait-time overlap
- event loop = scheduler
- CPU work blocks async

You are ready for Section 2.

2 SECTION 2 — ASYNC SYNTAX CLARITY (NO CONFUSION ALLOWED)

Goal of this section: Kill “async magic thinking”. Replace it with **mechanical understanding**.

If you misunderstand syntax, you will:

- write fake async
 - block the event loop
 - create bugs that don't crash but silently slow everything
-

CORE TRUTH (LOCK THIS FIRST)

Async syntax does not create concurrency. It only allows the event loop to take control.

Everything below exists to support this single idea.

2.1 `async def` — WHY THIS FUNCTION CAN “PAUSE”

1 Mental Model (Feynman)

`async def` does **not** mean:

- faster
- parallel
- background

It means:

“This function is *cooperative*. It agrees to give control back when it reaches `await`.”

An `async def` function:

- does **not** run immediately
- produces a **coroutine object**
- is **inert** until awaited or scheduled

Think:

- Normal function → executes now
 - Async function → returns a *promise of work*
-

2 Why This Exists (Real World Pain)

Without `async def`:

- Python has no safe pause points
- No controlled yielding
- No scheduler intervention

Async scraping needs:

- predictable pause points
 - safe switching
 - zero guessing about when code yields
-

3 Minimal Correct Syntax

```
async def fetch(url):  
    return url
```

What actually happens internally:

- Python creates a coroutine object
- No code inside runs yet

```
coro = fetch("https://example.com")  
# Nothing executed
```

Execution starts only when:

- `await fetch(...)`
 - `asyncio.create_task(fetch(...))`
 - `asyncio.gather(...)`
-

4 Failure Modes (Very Common)

Mistake 1: Thinking `async def` runs immediately

```
async def f():
    print("run")

f() # prints nothing
```

Mistake 2: Mixing sync and async mentally

```
async def f():
    requests.get(url) # blocks event loop
```

Result:

- Async shell
 - Sync core
 - Worst of both worlds
-

5 Thinking Exercises

Answer without running code:

```
async def f():
    print("A")
    await asyncio.sleep(1)
    print("B")

x = f()
print("C")
```

Correct order:

- C
- nothing else

Why:

- coroutine never awaited
-

6 Debug This Code

```
async def main():  
    fetch(url1)  
    fetch(url2)  
  
asyncio.run(main())
```

Bug:

- coroutines created
- never awaited
- zero execution

Correct mental fix:

- `async def` $\neq$ execution
- awaiting is mandatory

2.2 `await` — CONTROLLED STOP, NOT SLEEP

1 Mental Model (Feynman)

`await` means:

"I cannot proceed until this finishes. You (event loop) may run something else meanwhile."

It is:

- a **yield point**
- a **handoff**
- a **checkpoint**

It is NOT:

- sleep
- delay
- pause-the-program

2 Why This Exists (Real World Pain)

Without `await`:

- no yielding

- no interleaving
- async becomes sync

Async scraping needs:

- exact points where waiting occurs
- zero ambiguity about blocking

3 Minimal Correct Syntax

```
html = await fetch(url)
```

Internal sequence:

1. Task reaches await
2. Task suspends
3. Event loop schedules others
4. Awaited operation completes
5. Task resumes

4 Failure Modes

Mistake 1: Awaiting non-awaitable

```
await 5
```

Mistake 2: Forgetting await

```
fetch(url) # coroutine ignored
```

Mistake 3: Awaiting CPU work

```
await parse_html(html) # blocks
```

Result:

- event loop frozen

- async illusion broken
-

5 Thinking Exercises

Decide where `await` is legal:

- HTTP request → yes
- `asyncio.sleep` → yes
- `time.sleep` → no
- regex parsing → no
- `json.loads` → no

Rule:

Await only things that *wait externally*.

6 Debug This Code

```
async def f():
    data = fetch(url)
    return data
```

Bug:

- `data` is coroutine
- `fetch` never executed

Fix mentally:

- execution requires `await`
-

2.3 `asyncio.run()` — ENGINE IGNITION

1 Mental Model (Feynman)

`asyncio.run()` is:

“Start the event loop, run this coroutine, clean everything.”

It:

- creates loop

- runs main task
- waits until completion
- closes loop

You get **exactly one** per process.

2 Why This Exists

Before `asyncio.run()`:

- manual loop management
- resource leaks
- broken shutdowns

Async scraping demands:

- clean lifecycle
 - predictable teardown
 - no dangling sockets
-

3 Minimal Correct Syntax

```
async def main():  
    ...  
  
asyncio.run(main())
```

4 Failure Modes

Mistake 1: Nested `asyncio.run`

```
asyncio.run(main())  
asyncio.run(other())
```

Illegal.

Mistake 2: Using inside existing loop (common in notebooks)

Error:

- “event loop already running”

5 Thinking Exercise

Question:

Why can't we just call `await main()` at top level?

Answer:

- Python top-level is sync
- needs a loop owner

6 Debug This Code

```
def start():
    asyncio.run(fetch(url1))
    asyncio.run(fetch(url2))
```

Bug:

- loop created twice
- expensive
- illegal in async contexts

2.4 `asyncio.gather()` — CONCURRENCY AS A DECLARATION

1 Mental Model (Feynman)

`gather()` means:

"These tasks may run independently. Finish all before I continue."

It is:

- a **coordination primitive**
- not a thread spawner

2 Why This Exists

Without `gather`:

- manual task tracking
- fragile loops
- hard error handling

Scraping needs:

- batch control
- result aggregation
- failure tolerance

3 Minimal Correct Syntax

```
results = await asyncio.gather(  
    fetch(url1),  
    fetch(url2),  
    fetch(url3),  
)
```

Internal behavior:

- coroutines scheduled together
- event loop interleaves them
- waits for all to finish

4 Failure Modes

Mistake 1: Using gather in loop

```
for url in urls:  
    await asyncio.gather(fetch(url))
```

Zero concurrency.

Mistake 2: Ignoring exceptions

Default:

- first exception cancels rest

5 Thinking Exercises

What runs first?

```
await asyncio.gather(  
    slow(),  
    fast()  
)
```

Answer:

- both start immediately
 - completion order irrelevant
-

6 Debug This Code

```
tasks = [fetch(u) for u in urls]  
results = asyncio.gather(tasks)
```

Bug:

- passing list instead of unpacked coroutines
-

2.5 ILLEGAL PATTERNS (PYTHON ALLOWS, LOGIC BREAKS)

1 Async Function with No Await

```
async def f():  
    print("hi")
```

Effect:

- sync function wearing async costume
-

2 Blocking Calls Inside Async

```
async def f():  
    time.sleep(5)
```

Effect:

- freezes entire system

3 Over-Awaiting

```
await asyncio.gather(await fetch(url))
```

Effect:

- sequential execution
- wasted abstraction

4 Fire-and-Forget Without Tracking

```
asyncio.create_task(fetch(url))
```

Effect:

- unobserved failures
- silent data loss

DELIVERABLE 1 — SYNTAX → INTERNAL MECHANICS TABLE

Syntax	What Python Actually Does
<code>async def</code>	Creates coroutine factory
Calling async fn	Returns coroutine object
<code>await</code>	Suspends task, yields to loop
<code>asyncio.run</code>	Owns loop lifecycle
<code>gather</code>	Schedules coroutines together

DELIVERABLE 2 — BEGINNER MISTAKES CHECKLIST

If any is true, system is broken:

- Async function without await
 - requests inside async
 - time.sleep anywhere
 - gather inside loop
 - coroutine objects returned instead of data
 - no concurrency limits (next section)
-

SECTION 2 PASS CRITERIA (BRUTAL)

You pass only if:

- You can predict execution order without running code
- You know when code is *created* vs *executed*
- You can point to every **await** and justify it
- You never say “async magic”

If not, do not move to Section 3.

3 SECTION 3 — AIOHTTP BASICS (NETWORK LAYER)

Section intent: Teach you how to do **HTTP at scale without leaks, bans, or ghost bugs**. This is not about “using aiohttp”. This is about **network discipline**.

If this section is weak, everything above it collapses.

CORE TRUTH (LOCK THIS FIRST)

Async scraping lives or dies at the network layer. Most failures are not logic bugs — they are connection mistakes.

3.1 Why **requests** Fails at Scale

1 Mental Model (Feynman)

requests is like:

- calling people one by one
- using a new phone every time
- waiting silently until they answer

`aiohttp` is like:

- shared call center
- multiple calls in flight
- operator switches when someone is waiting

Key difference:

- **connection reuse**
 - **non-blocking sockets**
-

2 Why This Exists (Real World Pain)

Problems with `requests` in scraping:

- Each request blocks CPU
- New TCP connection per call (slow)
- No native concurrency
- Easy to overload your own machine

At scale (1000+ URLs):

- CPU idle time explodes
 - Total runtime balloons
 - OS socket limits hit
 - You mistake “Python slow” for “design wrong”
-

3 Minimal Correct Syntax (Contrast)

```
# requests (sync)
r = requests.get(url)
html = r.text
```

vs

```
# aiohttp (async)
async with session.get(url) as resp:
    html = await resp.text()
```

Critical difference:

- socket wait does not block event loop
 - response body read is awaitable
-

4 Failure Modes

- Using requests inside async → fake async
 - Creating new session per request → socket exhaustion
 - No timeout → hanging tasks forever
-

5 Thinking Exercise

Question:

Which is slower: 1 request taking 2s or 100 requests taking 2s each?

Wrong answer:

- "100 requests = 200s"

Correct async answer:

- ~2–5s depending on limits
-

6 Debug This Code

```
async def fetch(url):  
    r = requests.get(url)  
    return r.text
```

Bug:

- Blocks event loop
 - No concurrency
 - Async wrapper is meaningless
-

3.2 ClientSession — SHARED PHONE LINE (CRITICAL)

1 Mental Model (Feynman)

ClientSession =

- connection pool
- cookie jar
- DNS cache
- TCP reuse manager

It is **not** optional.

Creating a session per request is like opening a new browser for every click.

2 Why This Exists (Real World Pain)

Without session reuse:

- TLS handshake every time
- DNS lookup repeated
- Slow startup per request
- Servers flag abnormal behavior

With session reuse:

- Faster requests
 - Fewer sockets
 - More stable identity
-

3 Minimal Correct Syntax

```
async with aiohttp.ClientSession() as session:
    async with session.get(url) as resp:
        data = await resp.text()
```

Golden rule:

- **One session per scraping job**
 - Not per request
 - Not global forever
-

4 Failure Modes

Mistake 1: Session per URL

```
async def fetch(url):
    async with aiohttp.ClientSession() as s:
        async with s.get(url) as r:
            return await r.text()
```

Effect:

- destroys connection pooling
- slow
- suspicious traffic pattern

Mistake 2: Never closing session

Leads to:

- warnings
- leaked sockets
- OS-level failures

5 Thinking Exercise

Decide:

- 10k URLs → how many sessions?

Correct:

- 1 (or very few, segmented intentionally)

6 Debug This Code

```
session = aiohttp.ClientSession()

async def fetch(url):
    async with session.get(url) as r:
        return await r.text()
```

Bug:

- session created outside event loop
- lifecycle unmanaged
- cleanup uncertain

3.3 `async with` — WHY IT MATTERS (NOT STYLE)

1 Mental Model (Feynman)

`async with` =

- guaranteed cleanup
- even on exception
- even on timeout

It is:

- a **resource contract**
 - not syntactic sugar
-

2 Why This Exists

Network resources:

- sockets
- buffers
- file descriptors

If not released:

- system slows down
 - connections stall
 - scraper “mysteriously” hangs
-

3 Minimal Correct Syntax

```
async with session.get(url) as resp:
    text = await resp.text()
```

What it guarantees:

- response closed
 - connection returned to pool
-

4 Failure Modes

```
resp = await session.get(url)
text = await resp.text()
# forgot to close
```

Effect:

- connection leak
 - pool starvation
-

5 Thinking Exercise

Question:

Why does code work for 50 URLs but hang at 500?

Answer:

- leaked connections
 - pool exhausted
-

6 Debug This Code

```
async def fetch(url):
    resp = await session.get(url)
    return await resp.text()
```

Bug:

- resp never closed
-

3.4 STATUS CODE HANDLING (ASYNC REALITY)

1 Mental Model (Feynman)

HTTP status $\neq$ exception.

- 404 is data
- 429 is warning
- 500 is server pain

Async makes this harder because:

- failures are concurrent
 - silent unless logged
-

2 Why This Exists

Scrapers die because:

- they assume success
 - they parse error pages
 - they retry wrong things
-

3 Minimal Correct Handling

```
async with session.get(url) as resp:
    if resp.status != 200:
        return None
    return await resp.text()
```

4 Failure Modes

- Parsing 403 HTML as valid data
 - Retrying 404 endlessly
 - Ignoring 429 signals
-

5 Thinking Exercise

Decide behavior:

Status	Action
200	Parse
404	Skip
429	Slow down
500	Retry later

6 Debug This Code

```
html = await resp.text()
data = parse(html)
```

Bug:

- assumes HTML is valid
- status ignored

3.5 TIMEOUTS — WHAT ACTUALLY TIMES OUT

1 Mental Model (Feynman)

Timeout $\neq$ request timeout only.

There are:

- connection timeout
- read timeout
- total timeout

Async timeout = task cancellation.

2 Why This Exists

Without timeouts:

- tasks hang forever
- gather never completes
- scraper freezes silently

3 Minimal Correct Syntax

```
timeout = aiohttp.ClientTimeout(total=15)

async with aiohttp.ClientSession(timeout=timeout) as session:
    ...
```

4 Failure Modes

- No timeout → infinite wait
 - Too aggressive timeout → false failures
 - Treating timeout as crash
-

5 Thinking Exercise

Question:

Is timeout an error or a signal?

Correct answer:

- signal to adapt
 - not immediate failure
-

6 Debug This Code

```
async with session.get(url) as resp:  
    ...
```

Bug:

- no timeout
 - task may never return
-

DELIVERABLE 1 — MINIMAL REUSABLE `fetch()` FUNCTION

```
async def fetch(session, url):  
    try:  
        async with session.get(url) as resp:  
            if resp.status != 200:  
                return None  
            return await resp.text()  
    except asyncio.TimeoutError:  
        return None  
    except aiohttp.ClientError:  
        return None
```

Design principles:

- session passed in
- status checked
- failures contained
- no crashes

DELIVERABLE 2 — FAILURE-SAFE RESPONSE LOGIC

Rules:

- Network failure $\neq$ system failure
- Bad URL $\neq$ bad scraper
- Partial success is normal

SECTION 3 PASS CRITERIA (STRICT)

You pass only if:

- You use exactly one `ClientSession` per job
- You never leak responses
- You handle status codes explicitly
- You expect timeouts, not fear them
- Your fetch function **never crashes the system**

If any request can hang forever, you failed this section.

Proceed to Section 4 **only** after this is rock-solid.

SECTION 4 — CONCURRENCY CONTROL (THE HEART OF STAGE 2)

Concept Focus: Speed is not the goal. **Control is the goal.** Concurrency is a *resource governor*, not a performance trick.

0. CORE THESIS (READ THIS FIRST)

Async scraping without limits is **not advanced**. It is **reckless parallelism** that collapses under real-world constraints.

Concurrency control answers one question only:

“How many requests am I allowed to have *in flight* at the same time without breaking the system?”

If you cannot answer this, you are not doing async scraping. You are stress-testing servers unintentionally.

4.1 — WHY ASYNC WITHOUT LIMITS = BAN GENERATOR

1 Mental Model (Feynman)

Imagine a call center.

- Sync scraping → **one caller**, waits until call ends
- Async scraping → **many callers**, calling at once
- No concurrency limit → **everyone calls simultaneously**

What happens?

- Phone lines overload
- System flags abnormal traffic
- Calls drop
- Numbers get blocked

Async does not mean infinite capacity. It means **overlapping waiting time**, not ignoring limits.

2 Why This Exists (Real-World Pain)

Without limits, you trigger:

- HTTP 429 (Too Many Requests)
- Silent throttling (server slows you)
- IP bans (temporary or permanent)
- Connection resets
- Local resource exhaustion (file descriptors, sockets)

Common beginner illusion:

“Async is slow / unstable”

Reality:

You removed the brakes and blamed the engine.

3 Minimal Correct Syntax

✗ Dangerous pattern (unbounded concurrency)

```
tasks = [fetch(url) for url in urls]
results = await asyncio.gather(*tasks)
```

If `urls = 10_000`, you just launched **10,000 concurrent requests**.

✓ Correct pattern (bounded concurrency)

```
sem = asyncio.Semaphore(10)

async def limited_fetch(url):
    async with sem:
        return await fetch(url)
```

This guarantees:

- At most **10 active network calls**
 - Others wait politely
-

4 Failure Modes (What Breaks)

Failure	Why it happens
Random timeouts	Server queue overflow
429 errors	Rate-limit triggered
Local crash	Too many open sockets
"Hanging forever"	Event loop saturated
Inconsistent results	Partial task collapse

5 Thinking Exercises

- If each request takes ~500ms, how many concurrent requests can your machine *actually* manage?
- What happens if DNS resolution itself becomes the bottleneck?

- Why does increasing concurrency sometimes make total runtime worse?
-

6 Debug This Code

```
tasks = []
for url in urls:
    tasks.append(fetch(url))

await asyncio.gather(*tasks)
```

Question: Why does this code look async but behave like a denial-of-service attack?

4.2 — SEMAPHORE EXPLAINED (CLUB BOUNCER MODEL)

1 Mental Model (Feynman)

Semaphore = **club bouncer**

- Club capacity = concurrency limit
- People waiting = pending tasks
- Entry allowed only if capacity exists

No argument. No emotion. Pure control.

2 Why This Exists

Async systems lack natural brakes.

- Sync → blocking provides automatic limits
- Async → no blocking → unlimited spawning

Semaphore restores **pressure**.

3 Minimal Correct Syntax

```
sem = asyncio.Semaphore(5)

async def worker(url):
    async with sem:
        return await fetch(url)
```

Key facts:

- `async with sem` is atomic
- No two tasks exceed the limit
- Waiting tasks do not consume CPU

4 Failure Modes

- Creating semaphore inside function (each task gets its own)
- Using `await sem.acquire()` without `finally: sem.release()`
- Setting concurrency too high and blaming aiohttp

5 Thinking Exercises

- Why is semaphore better than `asyncio.sleep()` for control?
- Why should semaphore live at **system level**, not function level?

6 Debug This Code

```
async def fetch(url):
    sem = asyncio.Semaphore(10)
    async with sem:
        ...
```

Bug: Semaphore recreated per call → zero control.

4.3 — CHOOSING CONCURRENCY NUMBERS (NO GUESSING)

1 Mental Model

Concurrency is constrained by **three ceilings**:

1. Your machine
2. The network
3. The server

Lowest ceiling wins.

2 Real-World Pain

Blind choices like:

- “Let’s try 100”
- “Async is fast, right?”

Result:

- Faster bans
 - Slower completion
 - Unstable runs
-

3 Practical Heuristic (Start Here)

Target	Safe starting concurrency
Public site	3–5
Authenticated API	5–10
Your own server	20–50
Local test server	50+

Increase **only after measuring**.

4 Failure Modes

- CPU idle but network saturated
 - Network idle but server throttling
 - High success rate initially → collapse later
-

5 Thinking Exercises

- Why does optimal concurrency change over time?

- Why does night-time scraping succeed more easily?
-

6 Debug This Scenario

You increased concurrency from 10 → 50 Total runtime increased.

Why?

4.4 — CONCURRENCY × RATE LIMIT × SERVER TOLERANCE

1 Mental Model

Concurrency ≠ Rate.

- Concurrency: *how many at once*
- Rate: *how many per second*

Both matter.

2 Real-World Reality

You can have:

- Low concurrency + high rate (fast responses)
- High concurrency + low rate (slow server)

Servers measure patterns, not just volume.

3 Correct Combined Control

```
sem = asyncio.Semaphore(5)

async def controlled_fetch(url):
    async with sem:
        await asyncio.sleep(0.2) # rate shaping
        return await fetch(url)
```

This:

- Caps simultaneous load
 - Smooths request bursts
-

4 Failure Modes

- Semaphore without sleep → burst spikes
 - Sleep without semaphore → backlog explosion
-

5 Thinking Exercises

- Why do burst patterns trigger bans faster than steady flow?
 - Why is human-like pacing safer?
-

6 Debug This Code

```
await asyncio.sleep(1)
async with sem:
    ...
```

Why is this ineffective for rate control?

4.5 — TESTING SAFE LIMITS (SYSTEMATIC, NOT RANDOM)

1 Mental Model

Treat concurrency like load testing, not gambling.

2 Stepwise Method

1. Start at concurrency = 1
2. Measure:
 - Success %
 - Avg response time

3. Increase by +1

4. Stop when:

- Errors spike
- Latency worsens

3 Minimal Test Harness

```
for c in range(1, 11):  
    sem = asyncio.Semaphore(c)  
    run_test()
```

4 Failure Modes

- Changing concurrency mid-run
- Ignoring latency increase
- Measuring only total time

5 Thinking Exercises

- Why is 100% success not the only metric?
- Why does latency matter even if requests succeed?

6 Debug This Reasoning

"It didn't error, so it's safe."

Why is this false?

SECTION 4 — DELIVERABLES

☒ Semaphore-Controlled Fetch Loop (Reference Pattern)

```
sem = asyncio.Semaphore(5)  
  
async def fetch_safe(session, url):
```

```
async with sem:
    try:
        async with session.get(url, timeout=10) as resp:
            return await resp.text()
    except Exception as e:
        return None
```

☑ Concurrency Tuning Checklist

- Semaphore defined once
- Concurrency measured, not guessed
- Rate shaping included
- Errors logged, not ignored
- Latency monitored
- System survives partial failure

🧠 Final Mental Lock-In

Async scraping is not about speed. It is about **pressure regulation**.

If sync scraping is walking, async scraping is traffic engineering.

You do not remove traffic lights to move faster. You **optimize flow**.

💧 SECTION 5 — FAILURE HANDLING (REAL-WORLD ASYNC)

Core truth: Async systems do not “fail”; they **partially degrade**. If you don’t design for that, your scraper is fake-fast and production-useless.

0 SYSTEM POSITIONING (WHY THIS SECTION EXISTS)

In sync scraping, failure is loud and linear. In async scraping, failure is **quiet, concurrent, fragmented, and delayed**.

Your job is not to eliminate failure. Your job is to **contain, classify, and route failure** without stopping throughput.

1 WHY ASYNC FAILURES ARE DIFFERENT FROM SYNC

1.1 Mental Model (Feynman)

Sync world: One worker. One task. If it fails, everything stops. Easy to notice.

Async world: 100 workers. Some succeed. Some hang. Some timeout. Some return garbage. Some raise exceptions *after* others already finished.

Failure is **distributed**, not centralized.

Async failure is not a crash. Async failure is *noise in a system*.

1.2 Real-World Pain

Without failure handling:

- 1 timeout cancels 999 good results
- One bad HTML crashes parsing of all tasks
- A single 429 response poisons the whole run
- You don't know *what* failed or *why*

Clients don't care about your traceback. They care about **how much usable data you delivered**.

2 FAILURE TAXONOMY (NON-NEGOTIABLE)

If you don't classify failures, you can't control them.

2.1 Categories You MUST Separate

Failure Type	Meaning	Typical Cause
Timeout	Server didn't respond in time	Rate limits, slow backend
Network Exception	Connection died	DNS, proxy, SSL
HTTP Error	Server responded but rejected	403, 429, 500
Bad Data	Response OK but content unusable	CAPTCHA, empty HTML
Parser Error	Your logic failed	DOM changed

Failure Type	Meaning	Typical Cause
Logic Bug	Your code is wrong	Wrong assumptions

Each category demands a **different reaction**.

3 TIMEOUT vs EXCEPTION vs BAD DATA

3.1 Mental Model

- **Timeout** → "Server too slow"
- **Exception** → "Transport broke"
- **Bad Data** → "Server lied politely"

Treating them the same is amateur behavior.

3.2 Minimal Correct Syntax

```
import asyncio
import aiohttp

async def fetch(session, url):
    try:
        async with session.get(url,
            timeout=aiohttp.ClientTimeout(total=10)) as resp:
            text = await resp.text()

            if resp.status != 200:
                return {"url": url, "error": f"HTTP
{resp.status}"}

            if not text or len(text) < 100:
                return {"url": url, "error": "BAD_DATA"}

            return {"url": url, "html": text}

    except asyncio.TimeoutError:
        return {"url": url, "error": "TIMEOUT"}

    except aiohttp.ClientError as e:
        return {"url": url, "error": f"NETWORK: {e}"}
```

```
except Exception as e:
    return {"url": url, "error": f"UNKNOWN: {e}"}
```

Key rule: No exception escapes `fetch()`.

3.3 Failure Mode (What Breaks in Real Life)

- Raising exceptions inside async tasks → `gather()` explodes
 - Returning `None` → downstream crashes
 - Printing errors → useless at scale
-

4 RETRY STRATEGIES (WHEN AND WHEN NOT)

4.1 Mental Model

Retry is **not persistence**. Retry is **probability correction**.

If retrying won't change the probability, you're wasting bandwidth.

4.2 What Is Retry-Worthy?

Failure	Retry?	Why
Timeout	☑ Yes	Temporary congestion
Network error	☑ Yes	Flaky transport
429	☑ Yes (delayed)	Rate limit window
500	⚠ Limited	Server unstable
403	✗ No	Access blocked
Bad HTML	✗ Usually no	CAPTCHA / honeypot
Parser bug	✗ Never	Your fault

4.3 Minimal Retry Wrapper (Correct)

```

async def fetch_with_retry(session, url, retries=3, base_delay=1):
    for attempt in range(retries):
        result = await fetch(session, url)

        if "html" in result:
            return result

        error = result["error"]

        if error in ("TIMEOUT",) or error.startswith("NETWORK"):
            await asyncio.sleep(base_delay * (2 ** attempt))
            continue

        if error.startswith("HTTP 429"):
            await asyncio.sleep(5 + attempt * 2)
            continue

        return result # non-retryable

    return {"url": url, "error": "RETRIES_EXHAUSTED"}

```

4.4 Failure Modes

- Infinite retries → silent infinite loop
- Retrying 403 → IP burn
- Retrying parser errors → delusion

5 SKIP LOGIC vs CRASH LOGIC

5.1 Mental Model

Crash logic: "Something went wrong → stop everything"

Skip logic: "This unit failed → isolate → continue"

Async systems require **skip-by-default**.

5.2 Rule Table

Situation	Action
-----------	--------

Situation	Action
Single URL fails	Skip
10% URLs fail	Log
30% URLs fail	Slow down
70% URLs fail	Abort run
Parser breaks	Stop pipeline
Network flaky	Retry

5.3 Task Runner That Never Panics

```

async def run_tasks(tasks):
    results = []

    for task in asyncio.as_completed(tasks):
        try:
            result = await task
            results.append(result)
        except Exception as e:
            results.append({"error": f"TASK_CRASH: {e}"})

    return results

```

Why `as_completed` matters: You process results **even if others fail or hang**.

6 WHY "FAIL FAST" IS BAD IN SCRAPING

6.1 Mental Model

Fail-fast works when:

- State must be consistent
- Partial results are useless

Scraping is the opposite:

- Partial data is valuable
- Per-URL independence exists

Fail-fast in scraping is **cargo-cult engineering**.

6.2 Real Example

10,000 URLs

- 9,500 good
- 500 blocked

Fail-fast system → **0 data** Failure-tolerant system → **95% delivery**

Client chooses the second every time.

7 THINKING EXERCISES (DO NOT SKIP)

1. Why should parsing errors **never** trigger retries?
2. What happens if one task hangs forever inside `gather()`?
3. Why is returning error objects better than raising exceptions?
4. How would you detect a site-wide ban early?
5. When should a scraper intentionally stop itself?

Write answers. If you can't, you don't own this section.

8 DEBUG THIS CODE (HANDS-ON)

✗ Broken Code

```
results = await asyncio.gather(*tasks)
for r in results:
    process(r)
```

What's Wrong (List All)

- One exception cancels all tasks
- No failure classification
- No partial progress
- No timeout isolation

☒ Fixed Pattern

```
results = await asyncio.gather(*tasks, return_exceptions=True)

for r in results:
    if isinstance(r, Exception):
        log_error(r)
        continue
    process(r)
```

Still imperfect. Better is `as_completed()` with structured results.

9 OUTPUT ARTIFACT (YOU MUST INTERNALIZE)

Async scraping is not about avoiding failure. It is about designing a system where failure is expected, isolated, classified, and economically tolerated.

If you don't think like this, Stage 2 will break you.

SECTION 6 — ASYNC + PARSING BOUNDARY (HARD RULE)

Core rule: Async is for waiting. Parsing is for thinking. Never mix them.

This section fixes one of the most destructive beginner mistakes in async scraping: **making everything async without understanding the CPU-I/O boundary.**

1 Mental Model (Feynman)

Simple model

Think of a scraping system as a **factory pipeline**:

```
Internet (slow, unpredictable)
  ↓
Fetch HTML → Parse HTML → Extract Data → Save
```

- **Fetching** = waiting for the internet

- **Parsing** = CPU work inside your machine

Async is designed to **handle waiting**, not thinking.

If you try to make *thinking* async, you gain nothing and often lose stability.

2 Why This Boundary Exists (Real-World Pain)

Pain #1 — “My async scraper is slower than sync”

Cause:

- You async-wrapped CPU-heavy parsing
- Event loop gets blocked
- No real concurrency happens

Pain #2 — “My scraper randomly freezes”

Cause:

- Parsing blocks the event loop
- Other network tasks cannot resume
- Looks like a deadlock, but it's starvation

Pain #3 — “CPU usage is 100%, network idle”

Cause:

- Event loop is doing CPU work
- No task switching possible

Async solves I/O wait, not CPU load.

3 Minimal Correct Architecture (Non-Negotiable)

Correct pipeline

```
Async Zone (I/O bound)
-----
- HTTP requests
- Waiting for responses
- Retries
- Rate limiting

Sync Zone (CPU bound)
```

```
-----  
- BeautifulSoup / lxml parsing  
- Regex extraction  
- Data cleaning  
- Validation
```

Golden rule

Only the fetch layer is async. Everything else is sync unless proven otherwise.

4 Minimal Correct Syntax

Correct pattern

```
async def fetch(session, url):  
    async with session.get(url) as response:  
        html = await response.text()  
        return html # boundary here
```

```
def parse(html):  
    soup = BeautifulSoup(html, "lxml")  
    title = soup.select_one("title")  
    return title.text if title else None
```

```
async def main(urls):  
    async with aiohttp.ClientSession() as session:  
        tasks = [fetch(session, url) for url in urls]  
        html_pages = await asyncio.gather(*tasks)  
  
    results = []  
    for html in html_pages:  
        data = parse(html) # sync  
        results.append(data)
```

Boundary is explicit. No ambiguity.

5 Why Parsing Must Stay Sync (Deep Reasoning)

Event loop reality

- Python async uses **single-threaded event loop**
- Event loop switches tasks **only when `await` happens**
- Parsing libraries do **not** yield control

So when you do this:

```
async def parse(html):  
    soup = BeautifulSoup(html, "lxml")  
    return soup.title.text
```

You have created:

- An async function
- With **zero await points**
- That blocks the event loop entirely

This is worse than sync.

6 Illegal Patterns (Python Allows, Logic Breaks)

✗ Pattern 1 — Async parsing

```
async def parse(html):  
    soup = BeautifulSoup(html, "lxml") # blocks loop  
    return soup.text
```

Why wrong:

- No await
- Event loop frozen
- Other tasks starve

✗ Pattern 2 — Parsing inside fetch

```
async def fetch_and_parse(session, url):  
    async with session.get(url) as r:  
        html = await r.text()
```

```
soup = BeautifulSoup(html, "lxml") # BAD
return soup.title.text
```

Why wrong:

- CPU work inside async zone
- Concurrency illusion
- Scaling collapses

✗ Pattern 3 — “Everything async” mindset

```
async def clean(text): ...
async def extract(soup): ...
async def save(data): ...
```

Why wrong:

- Async is not a performance badge
- Async without I/O is useless
- Code becomes harder to debug

7 CPU vs I/O Boundary (Formal Definition)

I/O Bound

- Network calls
- Disk reads/writes
- API requests
- Waiting on external systems

Use async

CPU Bound

- HTML parsing
- Regex
- JSON transformation
- Data validation
- Hashing
- NLP preprocessing

Use sync

8 What Happens If You Break the Boundary

Symptom checklist

- Async slower than sync
- CPU pinned at 100%
- Requests stall randomly
- Timeouts increase with concurrency
- Logging timestamps freeze

Root cause

You blocked the event loop with CPU work.

9 Correct Advanced Pipelines

Pattern A — Simple (most cases)

```
async fetch → sync parse → sync save
```

Pattern B — Heavy parsing (advanced)

```
async fetch
  ↓
queue
  ↓
thread pool (CPU)
  ↓
save
```

Only use this if parsing is **proven** to be heavy.

10 Failure Modes (Real World)

Failure 1 — “But my parsing is fast”

False reasoning:

- Fast $\neq$ non-blocking
 - 5 ms $\times$ 10k pages = 50 seconds blocked loop
-

Failure 2 — “It works on small tests”

Async bugs scale non-linearly. Small tests hide starvation issues.

Failure 3 — “I used async everywhere, looks clean”

Clean syntax does not mean correct system behavior.

1.1 Thinking Exercises

1. Why does `await` not make CPU work non-blocking?
2. What happens if BeautifulSoup takes 20 ms per page with 500 concurrent fetches?
3. Why does increasing concurrency worsen performance when parsing is async?
4. Where exactly should the async–sync boundary be drawn in your current scraper?

Write answers. Do not skip.

1.2 Debug This Code (Hands-On)

Buggy code

```
async def fetch_parse(session, url):
    async with session.get(url) as r:
        html = await r.text()
        soup = BeautifulSoup(html, "lxml")
        return soup.title.text
```

What's wrong

- Parsing inside async function
 - Event loop blocked
 - Concurrency illusion
-

Corrected version

```

async def fetch(session, url):
    async with session.get(url) as r:
        return await r.text()

```

```

def parse(html):
    soup = BeautifulSoup(html, "lxml")
    return soup.title.text if soup.title else None

```

```

async def main(urls):
    async with aiohttp.ClientSession() as session:
        htmls = await asyncio.gather(
            *(fetch(session, u) for u in urls)
        )

    results = [parse(h) for h in htmls]
    return results

```

13 Hard Rules Summary

- Async ≠ faster by default
- Async is for **waiting**, not **thinking**
- Parsing inside async is a design bug
- One clear async–sync boundary
- If unsure, keep it sync

14 One-Line Law (Memorize)

Async waits. Sync thinks. Mixing them kills systems.

7 SECTION 7 — RATE LIMIT RESPECT (FREELANCER MODE)

Core thesis: Rate limiting is not a courtesy feature. It is a **system constraint** imposed by the server. Async scraping without rate-limit respect is not “fast”; it is **statistically guaranteed failure** at scale.

7.0 Mental Reset — What Rate Limiting Actually Is

Mental Model (Feynman)

A server is a **shared resource pool**. Each request consumes:

- CPU time
- Memory
- DB connections
- Network bandwidth

Rate limits exist to prevent **resource starvation** caused by aggressive clients.

Async scraping multiplies request *simultaneity*. If you don't slow it down intentionally, the server slows you down **violently** (timeouts, bans, IP blocks).

7.1 How Servers Detect Abuse

1. Mental Model

Servers don't "know" you are scraping. They observe **patterns**.

Bad scraper = statistically abnormal client.

2. Detection Signals (Real World)

Servers monitor combinations of:

- **Request frequency**
 - Requests per second/minute/hour
- **Burst behavior**
 - Sudden spikes (0 → 50 requests instantly)
- **Concurrency**
 - Too many open connections at once
- **Path behavior**
 - Sequential crawling at machine speed
- **Header fingerprints**
 - Missing / static headers

- **Error tolerance**
 - Client keeps retrying after 429 / 403
- **Session behavior**
 - No cookies, no state, no think time

You don't need to trigger all. **One strong signal is enough.**

3. Key Insight

Rate limiting is **probabilistic**, not binary.

You are not "safe" or "blocked". You are constantly moving on a **risk curve**.

7.2 Soft vs Hard Rate Limits

1. Mental Model

Think of two layers of defense:

Type	Analogy	Behavior
Soft limit	Speed breaker	Slower responses
Hard limit	Roadblock	Requests rejected

2. Soft Rate Limits

Symptoms:

- Response times slowly increase
- Random timeouts
- Occasional 429
- Inconsistent failures

What's happening:

- Server queueing your requests
- Throttling internally
- Lower priority assignment

Danger: Most scrapers ignore this phase and push harder.

That guarantees a hard block.

3. Hard Rate Limits

Symptoms:

- Consistent 429
- 403 / 401
- CAPTCHA
- Connection resets
- IP blacklisting (temporary or permanent)

What's happening: You crossed a statistical threshold.

Important: Once hard-limited, slowing down **after** doesn't help immediately.

7.3 Why Async Makes Rate Limits More Dangerous

1. Mental Model

Sync scraping fails *linearly*. Async scraping fails *exponentially*.

2. Comparison

Sync

- 1 request → wait → next
- Natural pacing
- Accidental politeness

Async

- 100 tasks scheduled instantly
- No natural delay
- Perfect abuse machine

Async removes friction. **You must reintroduce friction deliberately.**

7.4 Semaphore + Sleep = Two Different Controls

Most people confuse these. That's fatal.

7.4.1 Semaphore — Controls *Concurrency*

What it limits: "How many requests are in-flight at the same time"

What it does NOT limit: "How fast requests are sent over time"

Mental Model

Semaphore = number of open lanes on a bridge.

```
sem = asyncio.Semaphore(5)

async with sem:
    await fetch(url)
```

This ensures **at most 5 simultaneous requests**.

7.4.2 async sleep — Controls *Rate*

What it limits: “How often requests start”

```
await asyncio.sleep(0.5)
```

This ensures **spacing between requests**.

7.4.3 Why You Need Both

Problem	Semaphore	Sleep
Too many open connections	☑	✗
Too many requests per second	✗	☑
Burst detection	✗	☑
Server overload	☑	✗

Production rule: Semaphore without sleep = burst abuse Sleep without semaphore = slow but unstable

You need **both**.

7.5 Polite Scraper Timing Strategy (Production Grade)

1. Baseline Strategy (Safe Default)

- Concurrency: 3–5

- Delay: 300–800 ms
- Random jitter: $\pm 20\%$

2. Why Jitter Matters

Without jitter:

```
Request every 0.5s exactly
```

That is machine-perfect. Machine-perfect = suspicious.

With jitter:

```
delay = random.uniform(0.4, 0.7)
await asyncio.sleep(delay)
```

Now timing looks human-like.

3. Combined Pattern (Canonical)

```
sem = asyncio.Semaphore(5)

async def polite_fetch(url):
    async with sem:
        await asyncio.sleep(random.uniform(0.4, 0.7))
        return await fetch(url)
```

This is **freelancer-safe**.

7.6 Adaptive Slowing — Think, Don't Hammer

Static limits are fragile. Real systems adapt.

7.6.1 Mental Model

You probe the server gently and **listen**.

Server feedback channels:

- Response time
- Status codes
- Error frequency

7.6.2 Adaptive Signals

Signal	Meaning	Action
Avg response ↑	Soft throttling	Slow down
Timeouts ↑	Overload	Reduce concurrency
429 appears	Hard warning	Back off immediately
Success stable	Safe	Maintain

7.6.3 Simple Adaptive Logic

```
if status == 429:
    sleep *= 2
elif avg_response_time > threshold:
    sleep += 0.2
```

You don't need ML. You need **respectful feedback loops**.

7.7 Retry + Rate Limit Interaction (Critical Rule)

Retries amplify abuse if uncontrolled.

Rule:

Retry **after** slowing down, never at same speed.

Bad:

```
retry immediately
```

Correct:

```
await asyncio.sleep(backoff)
retry()
```

Backoff must increase, not stay constant.

7.8 Why This Matters for Freelancers

Client Reality

Clients don't care about:

- asyncio
- semaphores
- clever code

They care about:

- job completion
- IP safety
- repeatability
- zero downtime

A scraper that gets banned on day 2 is **unusable**.

Client-Ready Explanation (You Must Memorize)

"We intentionally limit speed so the system runs reliably for days instead of crashing in minutes. Faster isn't useful if it triggers blocks. This design protects your data pipeline long-term."

7.9 Failure Modes (Real World)

Common Mistakes

- Only using semaphore
 - Fixed sleep without jitter
 - Retrying 429 aggressively
 - Benchmarking on localhost logic
 - Assuming "no ban yet = safe"
-

Symptoms You Are Doing It Wrong

- Speed decreases over time
 - Random hangs
 - Inconsistent completion counts
 - Works today, fails tomorrow
-

7.10 Thinking Exercises

1. Why is 10 concurrency + 1s sleep still dangerous?
2. Why is 2 concurrency + no sleep still detectable?
3. Why does rate limiting matter more for long crawls than short ones?
4. Why does async exaggerate abuse signals?

Write answers without code.

7.11 Debug This Code (Hands-On)

Broken Code

```
async def fetch_all(urls):
    tasks = [fetch(url) for url in urls]
    return await asyncio.gather(*tasks)
```

What Breaks

- Instant burst
- No pacing
- No feedback
- Guaranteed detection

Fix Requirements

- Semaphore
- Sleep with jitter
- Retry backoff
- Observable behavior

If you cannot fix this confidently, you are not Stage 2.

Section 7 Summary (Non-Negotiable)

- Async increases **power**, not safety

- Rate limits are probabilistic defenses
- Semaphore controls *how many*
- Sleep controls *how often*
- Jitter avoids fingerprinting
- Adaptive slowing beats fixed numbers
- Freelancers optimize for **survival**, not peak speed

Async scraping is not about speed. It is about **earning the right to keep running**.

SECTION 8 — LOGGING IN ASYNC SYSTEMS

Concept Focus: Observability without chaos

Async scraping without logging is blindness at scale. Async scraping with naïve logging is *worse* than blindness: it creates false narratives.

This section builds **logging as a control system**, not as decoration.

8.0 SYSTEM GOAL (READ THIS FIRST)

Goal: You must be able to answer these *during* a run, not after it crashes:

- Which URLs are slow?
- Which tasks are stuck?
- Are failures random or systemic?
- Is the system degrading over time?
- Can I stop this run *confidently*?

If your logging can't answer these, it's noise.

8.1 WHY `print()` FAILS IN ASYNC

1 Mental Model (Feynman)

Think of async like **50 people talking at once on walkie-talkies**.

`print()` is everyone shouting into the same mic with:

- no timestamps
- no task identity

- no ordering guarantee

What you hear is **garbage**, not information.

2 Why This Exists (Real World Pain)

Real failures:

- Logs appear **out of order**, making fast tasks look slow
- A single stuck task hides among hundreds of prints
- You can't tell **which URL caused which error**
- Debugging requires rerunning expensive scrapes

In sync code, time order $\approx$ execution order. In async code, **time order is meaningless without context**.

3 Minimal Correct Syntax

Bad:

```
print("Fetching", url)
```

Still bad:

```
print(time.time(), url)
```

Correct *minimum*:

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s"
)

logging.info(f"Fetching {url}")
```

This gives:

- timestamps
- severity
- structured lines

Still not enough. Keep reading.

4 Failure Modes

- Logs lie by omission (no task identity)
 - You assume order implies causality
 - You miss silent hangs (no logs $\neq$ no problem)
 - Debugging turns into guesswork
-

5 Thinking Exercise

Answer honestly:

If 200 tasks run and one hangs forever, how will your current logging expose it?

If the answer is "I'll notice it's slow" $\rightarrow$ you're blind.

6 Debug This Code

```
async def fetch(url):
    print("start", url)
    await session.get(url)
    print("done", url)
```

Question: How many failure types produce *no* output at all?

8.2 TASK-AWARE LOGGING (CORE FIX)

1 Mental Model

Each async task is a **flight**. Logging without task identity is an airport screen without flight numbers.

You don't log *events*. You log **events + ownership**.

2 Why This Exists

Async introduces:

- interleaving
- partial completion
- orphaned tasks

Without task IDs:

- you cannot trace lifecycle
- you cannot reconstruct causality

3 Minimal Correct Syntax

Step 1: Name tasks

```
import asyncio
import logging

async def fetch(url):
    task = asyncio.current_task()
    task_name = task.get_name()

    logging.info(f"[{task_name}] start {url}")
```

Step 2: Create named tasks

```
task = asyncio.create_task(fetch(url), name=f"url-{i}")
```

Output becomes:

```
12:01:03 | INFO | [url-17] start https://example.com
```

Now logs are **traceable**.

4 Failure Modes

- Forgetting to name tasks → useless logs

- Reusing names → misleading traces
- Logging inside libraries without context

5 Thinking Exercise

You see this log:

```
ERROR timeout
```

What information is missing to fix the bug *without rerunning*?

6 Debug This Code

```
tasks = [asyncio.create_task(fetch(url)) for url in urls]
await asyncio.gather(*tasks)
```

Question: Why is post-mortem debugging almost impossible here?

8.3 TIMING, SLOW TASKS, AND PERFORMANCE SIGNALS

1 Mental Model

Async systems fail **gradually**, not explosively.

Speed degradation is the earliest warning signal. Logging time is how you see decay *before* bans and crashes.

2 Why This Exists

Common real-world issues:

- Server throttling increases latency slowly
- Network congestion appears as tail latency
- DNS or TLS stalls only some tasks

Without timing logs:

- you detect problems too late
- retries amplify damage

3 Minimal Correct Syntax

Per-URL timing

```
import time

start = time.perf_counter()
response = await session.get(url)
elapsed = time.perf_counter() - start

logging.info(f"[{task_name}] {url} took {elapsed:.2f}s")
```

Flag slow tasks

```
if elapsed > 5:
    logging.warning(f"[{task_name}] SLOW URL {url}")
```

4 Failure Modes

- Measuring only total runtime
- Ignoring tail latency
- Logging averages instead of outliers
- Treating “slow” as normal

5 Thinking Exercise

Two runs both finish in 10 minutes.

Run A:

- most URLs: 0.3s
- few URLs: 30s

Run B:

- all URLs: 0.6s

Which one is *healthier* and why?

6 Debug This Code

```
start = time.time()
await asyncio.gather(*tasks)
print("done in", time.time() - start)
```

Question: What critical failure pattern is invisible here?

8.4 DETECTING STUCK TASKS (THE SILENT KILLER)

1 Mental Model

A stuck async task is a **zombie**:

- not dead
- not progressing
- holding resources hostage

Your system may appear “running” forever.

2 Why This Exists

Causes:

- socket never returns
- server accepts but never responds
- bug before first log line
- forgotten await

Async does not protect you from logical deadlocks.

3 Minimal Correct Syntax

Heartbeat logging

```
async def watchdog(tasks):
    while True:
        alive = [t.get_name() for t in tasks if not t.done()]
        logging.info(f"Alive tasks: {len(alive)}")
        await asyncio.sleep(10)
```

Timeout enforcement

```
try:
    await asyncio.wait_for(fetch(url), timeout=15)
except asyncio.TimeoutError:
    logging.error(f"[{task_name}] HARD TIMEOUT {url}")
```

4 Failure Modes

- Infinite await without timeout
- Logging only on success
- Assuming `gather()` will return
- Ignoring unfinished tasks

5 Thinking Exercise

Your script runs for 40 minutes instead of 10. No errors. No logs. CPU near zero.

List **three** possible async causes.

6 Debug This Code

```
await asyncio.gather(*tasks)
logging.info("ALL DONE")
```

Question: Under what conditions does "ALL DONE" never print?

8.5 STRUCTURED LOGGING (OPTIONAL BUT ELITE)

1 Mental Model

Text logs are for humans. Structured logs are for **systems + humans**.

You want logs you can:

- filter
 - aggregate
 - reason about later
-

2 Why This Exists

At scale:

- text logs don't compress insight
 - searching becomes manual labor
 - trends are invisible
-

3 Minimal Correct Syntax

```
logging.info(  
    "fetch_complete",  
    extra={  
        "url": url,  
        "task": task_name,  
        "elapsed": elapsed,  
        "status": response.status  
    }  
)
```

Or JSON-style:

```
logging.info({  
    "event": "fetch_complete",  
    "url": url,  
    "task": task_name,
```

```
"elapsed": elapsed  
})
```

4 Failure Modes

- Mixing human prose and data
- Inconsistent keys
- Logging blobs instead of signals

5 Thinking Exercise

What log fields would you need to:

- prove politeness to a client?
- detect rate-limit pressure early?

6 Debug This Code

```
logging.info(f"{url} failed")
```

Question: What decision becomes impossible because of this line?

8.6 ASYNC LOGGING RULEBOOK (NON-NEGOTIABLE)

Memorize this.

- No `print()` in async systems
- Every log must answer **who, when, how long**
- Task identity is mandatory
- Time is a signal, not decoration
- Silence is a failure mode
- Logs exist to enable decisions, not comfort

SECTION 8 OUTPUT CHECK

You pass Section 8 if:

- You can identify a stuck task from logs alone
- You can explain slowdown without rerunning
- You can justify stopping a scrape mid-run
- Your logs form a *timeline*, not a diary

Async systems don't fail loudly. Logging is how you make them speak. Below are **full, exhaustive Stage-2-level notes for SECTION 9**. This is **engineering-grade**, not tutorial-grade. Read it as a systems document, not a blog.

9 SECTION 9 — PERFORMANCE PROOF (NO HAND-WAVING)

Core Thesis

Async scraping is not “fast by default.” It is measurable, conditional, and falsifiable.

If you cannot **prove** performance gains with clean experiments, your async system is untrusted and unsellable.

9.1 Why Performance Proof Exists (Real-World Pain)

The Pain Points

1. Illusion of speed

- Async *feels* faster because output arrives unordered.
- Human perception $\neq$ system throughput.

2. False comparisons

- Comparing:
 - sync with no retries
 - async with retries, logging, parsing
- This is invalid.

3. Client skepticism

- Clients do not care about **asyncio**.

- They care about:
 - total runtime
 - error rate
 - server friendliness

4. Over-optimization

- Async code that:
 - overwhelms servers
 - causes retries
 - increases bans
- Net result: **slower over 24 hours**

Performance proof exists to **kill delusion early**.

9.2 What "Performance" Actually Means (Define the Metric)

✗ Wrong Metrics

- "It feels faster"
- "CPU usage is high"
- "Requests per second" (alone)
- "Finished first few URLs quickly"

☑ Correct Metrics (You MUST define all)

Metric	Why it matters
Total wall-clock time	Client-visible performance
URLs processed	Throughput baseline
Success rate	Speed without correctness is fraud
Retry count	Hidden cost
Avg latency per URL	Efficiency
Slowest 5% URLs	Tail risk
Error distribution	Stability

Rule: If you cannot explain the slowest 5%, you don't understand the system.

9.3 Benchmarking Principles (Non-Negotiable Rules)

Rule 1 — Same Workload

- Same URLs
- Same headers
- Same retries
- Same parsing
- Same saving

No exceptions.

Rule 2 — Warm-up Runs

- First run is garbage:
 - DNS
 - TCP
 - SSL
 - Always discard first run.
-

Rule 3 — Fixed Environment

- Same network
- Same machine
- Same time window
- No VPN switching

Async speedups collapse under unstable networks.

Rule 4 — Measure End-to-End

You measure:

```
start_time → final result written
```

Not:

- fetch only
- partial tasks
- “when most finished”

9.4 Designing a Fair Sync vs Async Experiment

Step 1 — Define the Question

Bad question:

"Is async faster?"

Correct question:

"Under what concurrency does async outperform sync **without increasing failure rate?**"

Step 2 — Control Variables

Variable	Fixed
URLs	Same list
Parsing	Same function
Retries	Same logic
Headers	Same
Timeout	Same
Output	Same sink

Only difference:

- execution model

Step 3 — Choose Test Size

URLs	Why
< 50	Noise
100–300	Minimal
1k+	Real signal

Async advantages appear **after network latency dominates**.

9.5 Reference Benchmark Scripts

Sync Benchmark (Baseline)

```
import requests, time

def sync_fetch(urls):
    results = []
    start = time.perf_counter()

    for url in urls:
        try:
            r = requests.get(url, timeout=10)
            results.append(r.status_code)
        except Exception:
            results.append(None)

    return {
        "time": time.perf_counter() - start,
        "count": len(results),
        "success": sum(1 for r in results if r == 200)
    }
```

This is your **truth anchor**.

Async Benchmark (Controlled)

```
import aiohttp, asyncio, time

async def fetch(session, url, sem):
    async with sem:
        try:
            async with session.get(url, timeout=10) as r:
                return r.status
        except:
            return None

async def async_fetch(urls, concurrency):
    sem = asyncio.Semaphore(concurrency)
    start = time.perf_counter()

    async with aiohttp.ClientSession() as session:
        tasks = [fetch(session, u, sem) for u in urls]
        results = await asyncio.gather(*tasks)

    return {
```

```
"time": time.perf_counter() - start,
"count": len(results),
"success": sum(1 for r in results if r == 200)
}
```

Note: No logging. No parsing. No retries. This isolates **network behavior**.

9.6 Interpreting Results (This Is Where Most Fail)

Example Output

Mode	Time	Success
Sync	120s	980
Async (10)	40s	975
Async (50)	22s	910
Async (100)	18s	780

Correct Interpretation

- Async(10): **valid improvement**
- Async(50): faster but unstable
- Async(100): unacceptable

Critical Insight

Max speed $\neq$ optimal speed

Optimal point is:

- before success rate collapses
- before retries spike
- before server response degrades

This is **control theory**, not coding.

9.7 Explaining Speedup Logically (No Marketing)

✗ Bad Explanation

"Async is faster because it runs in parallel."

Wrong. Misleading. Amateur.

☒ Correct Explanation Template

"The speedup comes from overlapping network wait times. Each request spends most of its life waiting on the server. Async allows us to utilize that idle time safely within controlled concurrency limits."

Clients understand this.

9.8 Common Benchmarking Traps

Trap 1 — Measuring Task Creation Time

Async task creation is cheap. Network I/O is not.

Trap 2 — Ignoring Tail Latency

Average hides pain.

Always inspect:

- slowest URLs
 - retries per URL
-

Trap 3 — Ignoring Failure Cost

Retries consume:

- time
- IP reputation
- server goodwill

Speed that causes retries is **fake speed**.

Trap 4 — Comparing Different Retry Logic

Async often retries silently.

Sync often crashes.

That is not a fair test.

9.9 Performance Proof Checklist (Hard Gate)

You may claim async superiority only if:

- ☐ Same workload
- ☐ Same logic
- ☐ Same output
- ☐ ≥ 3 runs
- ☐ Success rate $\geq$ sync
- ☐ Concurrency justified
- ☐ Tail latency analyzed
- ☐ Explanation is non-magical

Miss one $\rightarrow$ claim invalid.

9.10 Mental Model Summary

Async performance is:

```
(Not speed)
= Controlled overlap
+ Measured throughput
+ Stable success rate
```

If you cannot prove all three, **you do not own the system.**

One-Paragraph Output Artifact (Mandatory)

Async scraping performance is not about raw speed but about controlled overlap of I/O wait times. Proper benchmarking requires identical workloads, fixed environments, and end-to-end measurement of time, success rate, and tail latency. Real gains appear only when concurrency is tuned below failure thresholds. Any speedup that increases retries, errors, or instability is fake performance and collapses at scale.

This completes **SECTION 9** at a **systems-engineer level**.

10 SECTION 10 — FREELANCE READINESS SIMULATION

Async scraping as a defensible system, not a trick

SECTION GOAL (NON-NEGOTIABLE)

This section tests **system ownership**.

Not:

- “Can you write async code?”

But:

- Can you **sell, defend, scope, limit**, and **survive** a real client job?

If you cannot explain *why* your async system behaves the way it does under pressure, you are not Stage-2 ready.

1 THE 10,000 URL THOUGHT EXPERIMENT (REALITY CHECK)

Scenario

Client says:

“We have ~10,000 product URLs. We need all data within a few hours. No bans. No downtime.”

Your job is not to code. Your job is to **reason**.

Mental Decomposition (Step-by-step)

Wrong beginner thinking

```
10k URLs  
→ async  
→ fast  
→ done
```

Correct system thinking

```
10k URLs  
→ server tolerance unknown
```

```
→ rate limits unknown
→ failure probability non-zero
→ retries needed
→ logs needed
→ partial delivery acceptable
```

First-principles breakdown

Variable	Question you must answer
URLs	Are they same domain or multiple?
Server	CDN? Cloudflare? Self-hosted?
Auth	Public or logged-in?
Data	Static HTML or API-backed?
SLA	Is partial data acceptable?
Time	"Fast" means what? Minutes? Hours?

Until these are answered, **speed discussion is illegal**.

Back-of-the-envelope math (what clients respect)

Assume:

- Avg response time: **500 ms**
- Safe concurrency guess: **10**
- URLs: **10,000**

```
10 requests every 0.5 sec
→ 20 req/sec
→ 10,000 / 20 = 500 sec ≈ 8.3 minutes
```

Now add:

- retries
- slow URLs
- pauses
- failures

Final estimate you tell client

“~15–25 minutes with conservative settings, without bans.”

This is credibility.

2 CLIENT Q&A SCENARIOS (DEFENSE MODE)

Q1: “Why not run 500 concurrent requests?”

Correct answer

- Because concurrency $\neq$ throughput
- Servers throttle, queue, or ban
- Beyond a point, error rate rises faster than speed

One-liner

“High concurrency increases *failure density*, not speed.”

Q2: “Can you make it even faster?”

Correct response structure:

1. Ask what they are willing to sacrifice:

- reliability
- politeness
- IP reputation

2. Explain tradeoff explicitly

Professional answer

“Yes, by increasing concurrency, but failure rate and ban risk rise. I recommend stability-first unless speed is business-critical.”

Never promise speed without cost.

Q3: “What happens if some URLs fail?”

Wrong answer:

“The script stops.”

Correct answer:

"Failures are isolated, logged, retried if meaningful, and skipped if toxic. You still get usable output."

Clients want **progress, not perfection**.

Q4: "What if the site blocks us?"

You must say **before** they ask:

- IP rotation is a separate scope
- Proxies cost money
- Async does not bypass detection

Boundary statement

"Async improves efficiency, not anonymity."

3 EXPLAINING TRADEOFFS IN SIMPLE LANGUAGE

Async explained to non-technical client

Bad explanation:

"It uses coroutines and event loops."

Correct explanation:

"Instead of waiting for one page to respond before asking the next, we politely ask multiple pages at once, but with a strict limit so the server doesn't get overwhelmed."

Speed vs Stability analogy (use this)

"It's like multiple checkout counters in a store. Too few is slow. Too many causes chaos."

What async DOES give them

- Predictable completion time
 - Controlled server load
 - Graceful degradation
 - Logs + proof of work
-

What async does NOT guarantee

- Zero bans
- Infinite speed
- Perfect data
- Immunity to bad servers

Saying this upfront prevents disputes.

4 WHEN **NOT** TO USE ASYNC (THIS IS A TEST)

Async is not a default.

Do NOT use async when:

Case	Why
<100 URLs	Sync is simpler and safer
Heavy JS rendering	Browser is bottleneck
CPU-heavy parsing	Async gives zero benefit
Strict per-request logic	Concurrency complicates correctness
One-off scripts	Maintenance cost > benefit

If you recommend sync when appropriate, clients trust you more.

5 SYSTEM DESIGN ANSWER (WHAT YOU SAY IN INTERVIEWS)

Question:

"Design a scraper for large-scale data collection."

High-level answer (structure matters)

1. **Fetch Layer (Async, limited)**

- aiohttp
- semaphore
- timeouts

2. **Failure Layer**

- retries for transient errors
- skip hard failures

3. **Parse Layer (Sync)**

- deterministic
- CPU-safe

4. **Storage Layer**

- incremental writes
- crash-safe

5. **Observability**

- logs
- metrics
- progress tracking

Say this calmly. No jargon excess.

6 COMMON FREELANCER FAILURES (LEARN FROM OTHERS' PAIN)

Failure 1: Overpromising speed

Outcome:

- bans
- incomplete data
- refunds

Failure 2: No logs

Outcome:

- cannot prove work
- cannot debug
- client distrust

Failure 3: One giant `gather()`

Outcome:

- memory spikes
- silent failures

- no recovery

Failure 4: Treating async as magic

Outcome:

- fragile system
- panic debugging
- career damage

7 CLIENT-READY EXPLANATION (DELIVERABLE)

You should be able to say this **verbatim**:

"This scraper is designed to run multiple requests concurrently, but under strict limits. That allows us to collect data faster while respecting server capacity. Failures are expected and handled individually, so you receive maximum usable data even if some pages fail. Speed is controlled, not forced."

If you cannot say this confidently, you are not ready.

8 FINAL SELF-CHECK (PASS / FAIL)

You pass Section 10 only if:

- You can **estimate time** before coding
- You can **justify concurrency numbers**
- You can **explain failure calmly**
- You can **say no to async when needed**
- You can **defend every design choice**

Async scraping is not a flex.

It is **operational responsibility**.

🌟 FINAL SECTION — THINKER MODE CERTIFICATION (STAGE 2 COMPLETE)

This section is not a recap. This is a **cognitive lock-in**. If these notes make sense *without* rereading previous sections, Stage 2 is internalized.

🧠 CORE IDENTITY SHIFT (NON-NEGOTIABLE)

You are no longer a “scraper writer”.

You are a **distributed system operator**.

Async scraping is not Python syntax. It is **load orchestration under uncertainty**.

If this mental shift does not happen, everything you build after this will collapse at scale.

THE FOUR GUARANTEES OF REAL-WORLD ASYNC SYSTEMS

Every async scraping system, without exception, obeys these rules:

1. **Concurrency always outruns control if unchecked**
2. **Failures are partial, silent, and non-linear**
3. **Speed amplifies mistakes**
4. **Debugging cost grows exponentially with chaos**

Stage 2 exists to enforce control against these guarantees.

CERTIFICATION CRITERIA — WHAT “STAGE 2 COMPLETE” ACTUALLY MEANS

You are Stage 2 complete **only if all four dimensions below are true simultaneously**.

CONCURRENTNESS IS INTENTIONAL, NOT ACCIDENTAL

What beginners do

- Launch 1000 tasks because “async can handle it”
- Equate `gather()` with safety
- Treat semaphores as optional

What Stage 2 thinkers do

- Decide concurrency **before** writing code
- Can justify a number like `limit=12` verbally
- Change concurrency based on:
 - domain
 - endpoint
 - time of day

- observed latency

Internal rule

If you cannot explain *why* your concurrency number exists, it is wrong.

Diagnostic self-test

You must be able to answer **without hesitation**:

- Why is concurrency 10 and not 20?
- What breaks if I double it?
- What improves if I halve it?

If answers are vague → Stage 2 incomplete.

2 FAILURE IS EXPECTED, MODELED, AND ROUTED

Async truth

Failure is not an exception. Failure is a **data stream**.

In async systems:

- Some tasks succeed
- Some hang
- Some timeout
- Some return garbage
- Some fail silently

All at the same time.

Stage 1 reaction

- Crash on first exception
- Retry blindly
- Lose task identity
- Panic-debug

Stage 2 reaction

- Categorize failures:
 - transport failure
 - server rejection
 - client timeout

- malformed response
- Decide routing:
 - retry
 - skip
 - log-only
 - downgrade speed

Internal rule

If one URL fails, the system must continue without emotional reaction.

Certification requirement

You must be able to say:

- Which failures are acceptable
- Which failures are terminal
- Which failures trigger slowdown
- Which failures are ignored

If “everything retries” → incorrect.

3 YOU DEBUG ASYNC SYSTEMS WITHOUT PANIC

Async debugging reality

You never debug “a bug”. You debug **timelines**.

Stage 1 mindset

- “Why is nothing printing?”
- “Why is it stuck?”
- “Why did it suddenly stop?”

Stage 2 mindset

- Which tasks are still alive?
- Which tasks are slow?
- Which tasks never released semaphore?
- Which await never returned?

Observable signals you rely on

- timestamps

- task IDs
- duration per URL
- active vs completed counts

Internal rule

If you cannot observe it, you do not control it.

Certification test

Given a stuck async run, you must know **where to look first**:

1. semaphore acquisition
2. network timeout
3. await chain
4. task scheduling

Random print-debugging = fail.

4 EVERY ASYNC LINE HAS A DEFENSIBLE REASON

This is the hardest requirement.

Stage 1 code

```
async def fetch():  
    ...
```

Written because tutorial said so.

Stage 2 code

```
async def fetch():  
    # async because network I/O blocks
```

Every line must survive interrogation.

You must be able to answer:

- Why async here?
- Why sync here?
- Why await here?

- Why not gather here?
- Why semaphore outside, not inside?

Internal rule

Async without justification is technical debt.

If you cargo-cult async → Stage 2 incomplete.

SYSTEM THINKING CHECKLIST (PASS / FAIL)

You pass Stage 2 **only if all answers are YES**.

- Do you design concurrency before coding?
- Do you treat failures as expected outcomes?
- Do you log with task awareness?
- Do you know when async is harmful?
- Do you separate I/O speed from CPU work?
- Do you slow down deliberately when signals appear?
- Do you know when **not** to use async?

One "NO" = revisit Stage 2.

COMMON DELUSIONS STAGE 2 ELIMINATES

If you believe any of these, certification is invalid:

- "Async always faster"
- "More concurrency = more speed"
- "Retries solve everything"
- "Timeouts are rare"
- "Logging is optional"
- "Parsing async is cleaner"
- "If it works locally, it works at scale"

These beliefs collapse systems.

THE FINAL MENTAL MODEL (LOCK THIS IN)

Async scraping is **traffic management**.

- URLs are vehicles
- Event loop is the intersection

- Semaphore is the traffic light
- Timeouts are stalled cars
- Retries are rerouting
- Logging is CCTV
- Rate limits are police presence

Your job is **flow**, not speed.

STAGE 2 OUTCOME (OBJECTIVE, NOT EMOTIONAL)

After Stage 2:

- You can scrape 10k URLs without fear
- You can explain your system to a client
- You can slow down without ego
- You can prove performance, not claim it
- You can debug under pressure

This is the minimum bar for professional async scraping.

FINAL CERTIFICATION STATEMENT

If you truly completed Stage 2:

You do not ask

"How fast can I go?"

You ask

"How fast can I go **without losing control**?"

That question alone proves certification.

Tum coder nahi ho. Tum traffic controller ho.